Cykl 1: Zgłoszenie online (Scenariusz 5)

Cel: klient może zgłosić usterkę i dostać numer zlecenia.

1. **Baza danych:** `FORMULARZ_ZGLOSZENIA`, `ZLECENIE` (z `numer`, `formularz_id` UQ, FK do urządzenia i kategorii). Numer zlecenia musi być unikalny i trudny do odgadnięcia (nie sekwencyjny), bo to jedyny „klucz" klienta.
2. **Backend:** `FormController` (`submitForm`), `ValidationService` (`validateData`), zapis przez `OrderRepository`. Utworzenie zlecenia w stanie NEW. Ścieżka błędu: walidacja zwraca listę błędów bez zapisu.
3. **Powiadomienia:** serwis e-mail (na razie może to być stub/logowanie) wysyłający numer zlecenia.
4. **Frontend:** ekrany 7.1 (strona główna), 7.3 (formularz: typ zlecenia, typ urządzenia, producent i model, e-mail, opis) i 7.4 (potwierdzenie). Brakuje w formularzu imienia, nazwiska i telefonu, które są w tabeli `KLIENT`, więc trzeba je dodać lub zdecydować, że są opcjonalne.
5. **Testy:** poprawne zgłoszenie, brak wymaganych pól, nieprawidłowy e-mail.

Gotowe, gdy: wypełnienie formularza tworzy rekord w bazie ze statusem NEW i klient dostaje numer.

Cykl 2: Sprawdzanie statusu i anulowanie (Scenariusz 6)

Cel: pełna samoobsługa klienta od strony strony WWW.

1. **Baza danych:** tabela `ANULOWANIE` (`powod`, `zlecenie_id` UQ).
2. **Backend:** endpoint statusu (numer → urządzenie + status) oraz `CancellationService` (`validateOrderId`, `cancelOrder`). Anulowanie jest dozwolone tylko ze stanu NEW, a dla DELIVERED i wyższych zwraca czytelny błąd. Zapis do `ANULOWANIE` i zmiana statusu na CANCELLED powinny odbywać się w jednej transakcji.
3. **Frontend:** ekrany 7.1 (pole ID i „Sprawdź"), 7.5 (wynik wyszukiwania z przyciskiem „Anuluj zamówienie"), 7.6 (potwierdzenie TAK/NIE) i 7.7 (nieprawidłowy numer). Warto dodać ochronę przed zgadywaniem numerów (rate limiting).
4. **Testy:** anulowanie w NEW, odrzucenie w DELIVERED/IN_PROGRESS/COMPLETED, ponowne anulowanie tego samego zlecenia, nieistniejący numer.

Gotowe, gdy: klient sprawdza status i anuluje zlecenie, a przy późniejszym stanie dostaje komunikat odmowy.

Cykl 3: Panel technika i przyjęcie sprzętu (Scenariusz 1)

Cel: technik widzi zlecenia i potwierdza fizyczne dostarczenie.

1. **Baza danych:** `TECHNIK` oraz konta użytkowników/role (nie ma ich w ER, więc trzeba je dodać).
2. **Backend:** logowanie techników i autoryzacja ról, wyszukiwanie zlecenia po ID, zmiana NEW → DELIVERED, lista zleceń z filtrem po statusie.
3. **Powiadomienie:** potwierdzenie przyjęcia urządzenia wysyłane do klienta.

4. **Frontend:** `PanelTechnika` (`displayOrder`): wyszukiwarka, lista, szczegóły zlecenia. Zlecenia CANCELLED widoczne jako anulowane, żeby technik wiedział, że klient się nie zjawi (krok 4 Scenariusza 6, tu realizowany po stronie panelu).
5. **Testy:** próba przyjęcia zlecenia CANCELLED lub już DELIVERED, dostęp bez roli technika.

Gotowe, gdy: technik loguje się, znajduje zlecenie po numerze i oznacza je jako DELIVERED.

Cykl 4: Diagnoza i kosztorys (początek Scenariusza 2)

Cel: technik tworzy kosztorys, a klient go otrzymuje.

1. **Baza danych:** `KOSZTORYS` (`kwota`, `status_akceptacji`, `zlecenie_id` UQ) i `NAPRAWA` (`opis`, `czas_naprawy`, `technik_id`).
2. **Backend:** `RepairService.createEstimate()` i `updateOrderStatus()`. Rozpoczęcie diagnozy to DELIVERED → IN_PROGRESS. Wysłanie kosztorysu mailem z bezpiecznym linkiem do podjęcia decyzji.
3. **Frontend:** formularz diagnozy i wyceny w panelu technika. Panel klienta do przeglądu kosztorysu nie ma wireframe'a w dokumentacji, więc trzeba go zaprojektować (proponuję dołożyć go do ekranu 7.5).
4. **Testy:** tworzenie kosztorysu tylko w IN_PROGRESS, kwoty ujemne i zerowe, wygasanie linku.

Gotowe, gdy: po diagnozie klient dostaje maila z kosztorysem i widzi go na stronie.

Cykl 5: Decyzja, płatność i naprawa (dokończenie Scenariusza 2)

Cel: klient akceptuje lub odrzuca kosztorys, płaci, a technik wykonuje naprawę.

1. **Baza danych:** `PLATNOSC` (`kwota`, `status`, `data_platnosci`, `zlecenie_id` UQ).
2. **Backend:**
    - akceptacja kosztorysu → płatność → naprawa,
    - odrzucenie kosztorysu → od razu COMPLETED (wydanie bez naprawy),
    - `performRepair()`, po którym status zmienia się na COMPLETED.
3. **Integracja płatności:** bramka płatnicza (Przelewy24, Stripe, BLIK itp.) za interfejsem, z początkową implementacją testową/sandbox. Płatność trzeba obsłużyć asynchronicznie (webhook).
4. **Frontend:** akceptacja/odrzucenie kosztorysu i przejście do płatności po stronie klienta, a w panelu technika widok „do naprawy" i przycisk zakończenia naprawy.
5. **Testy:** ścieżka akceptacji, ścieżka odrzucenia, nieudana płatność, podwójne kliknięcie „Zapłać".

Gotowe, gdy: zlecenie przechodzi przez całą ścieżkę DELIVERED → IN_PROGRESS → COMPLETED, z płatnością i bez.

Cykl 6: Części i integracja z hurtownią (Scenariusz 7)

Cel: automatyczne zamawianie brakujących części.

1. **Baza danych:** `CZESC` (`nazwa`, `cena`, `dostepnosc`) i `ZLECENIE_CZESC` (`ilosc`, `status_zamowienia`).
2. **Backend:**

    - `PartsService.checkAvailability()` i `orderParts()`,
    - brak części ustawia status WAITING_FOR_PARTS,
    - `WarehouseApiClient.sendOrder()` wysyła zamówienie do hurtowni,
    - po zaktualizowaniu dostępności zlecenie wraca do IN_PROGRESS.
3. **Integracja:** adapter do API hurtowni. Nie wiadomo, jakie to API, więc na start proponuję mock/fake hurtowni z identycznym interfejsem, a prawdziwy adapter dopiero po wyborze dostawcy.
4. **Frontend:** w panelu technika dodawanie wymaganych części do zlecenia, widok statusu zamówienia i przycisk „Część dotarła" (potwierdzenie dostawy).
5. **Testy:** część dostępna na magazynie, część niedostępna, ponowienie wysyłki przy błędzie hurtowni (retry), powrót ze statusu WAITING_FOR_PARTS.

Gotowe, gdy: brak części automatycznie wstrzymuje zlecenie i wysyła zamówienie, a dostawa je odblokowuje.

Cykl 7: Wydanie sprzętu i rekompensaty (Scenariusz 3)

Cel: formalne zamknięcie zlecenia.

1. **Baza danych:** `ZWROT` (`kwota`, `powod`, `typ_zwrotu`, `platnosc_id`) oraz mechanizm archiwizacji zleceń (znacznik lub osobna tabela).
2. **Backend:** `BillingService` (weryfikacja rozliczenia, zamknięcie zlecenia, archiwizacja), `CompensationService.calculateCompensation()` (w diagramie 5.2 jest powiązany z klasą `Order`, którą trzeba ujednolicić ze `Zlecenie`). Rekompensata jest wypłacana przed zamknięciem zlecenia. Przejście COMPLETED → stan końcowy.
3. **Frontend:** w panelu technika akcje „Wydaj sprzęt" i „Zgłoś uszkodzenie / zwrot środków" z typem i powodem.
4. **Testy:** wydanie standardowe, wydanie po odrzuconym kosztorysie, wydanie z rekompensatą, zwrot przekraczający kwotę płatności.

Gotowe, gdy: zlecenie można zamknąć, w razie szkody wypłacić rekompensatę, a dane trafiają do archiwum.

Cykl 8: Rozliczenie podatkowe (Scenariusz 4)

Cel: zautomatyzowane okresowe rozliczenia.

1. **Baza danych:** `STAWKA_PODATKU` (z seedem stawek) i `DANE_PODATKOWE` (`nip`, `kwota_netto`, `okres`, `stawka_id`).
2. **Backend:** `TaxSettlementService.prepareTaxData()` i `sendReport()`. Agregacja zamkniętych i opłaconych zleceń z okresu, scheduler (np. cron) uruchamiający proces, zapis UPO.
3. **Integracja:** `TaxInterface.send()` i `GovernmentApiClient` (`connect`, `authorize`). Dokumentacja sama zaznacza niepewność co do API, więc najpierw trzeba zbadać, jaki format i kanał wymaga polskie prawo dla tego rodzaju rozliczeń.
4. **Plan awaryjny:** eksport danych do pliku (CSV/XML) do ręcznego złożenia, jeśli automatyczna wysyłka okaże się niemożliwa.

5. **Testy:** poprawność agregacji kwot, pominięcie zleceń niezamkniętych i nieopłaconych, błąd po stronie urzędu, brak podwójnego rozliczenia tego samego okresu.

Gotowe, gdy: scheduler generuje poprawny raport za okres i zapisuje potwierdzenie albo udostępnia eksport ręczny.

Cykl 9: Stabilizacja i wdrożenie

Cel: gotowość produkcyjna.

1. **Testy end-to-end:** pełna ścieżka zlecenia, od formularza do wydania, plus wszystkie ścieżki alternatywne (anulowanie, odrzucony kosztorys, brak części, rekompensata).
2. **Bezpieczeństwo:** RODO (dane osobowe klientów, polityka retencji), nagłówki bezpieczeństwa, rate limiting, audyt dostępu do panelu technika.
3. **Strona „Oferta zakładu" (7.2):** w dokumentacji to na razie placeholder. Wypełnienie treścią.
4. **Dokumentacja i operacje:** logi, monitoring, backup bazy, instrukcja dla techników.
5. **Wdrożenie:** środowisko produkcyjne, migracje, pilot z kilkoma prawdziwymi zleceniami.